

P3↔P1 Tool Agnosticism Inheritance

Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

May 15, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes commitments across the CKS theory trilogy: “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026), and “Inter-Self Coordination via Shared Substrate / Full Aspect Integration” (Li, April 2026).

Abstract

Paper 1 of the CKS trilogy established tool agnosticism as an architectural commitment: the governance architecture works on any compliant host platform — any environment satisfying three minimal requirements — because governance authority resides in the authored substrate content, not in the technical implementation that hosts it. Paper 3 extends this commitment in two directions at inter-Self scope. First, the shared substrate through which two or more CKS-governed Selves coordinate can be hosted on any compliant technical platform; it need not use the same infrastructure as either participating Self’s home substrate. Second, Paper 3 introduces domain agnosticism as the organizational content analog of tool agnosticism: the same architecture operates in any organizational domain — healthcare, finance, operations, or any other — because governance requirements are addressed through substrate configuration, not through architectural modification. This note formalizes that inheritance relationship, identifies what is preserved from Paper 1 and what is extended at inter-Self scope, and states the prior-art closure: any implementation achieving shared-substrate platform independence or domain-agnostic inter-organizational coordination is implementing commitments that trace to Paper 1’s tool-agnosticism claim.

1. Inheritance Identification

Source commitment: Paper 1, Claim 5 — tool agnosticism at intra-Self (cell) scope.

Extending commitments in Paper 3: (a) platform independence for the shared substrate at inter-Self scope; (b) domain agnosticism as the content analog of tool agnosticism, applying to organizational domain rather than technical platform.

The inheritance relationship is direct: both of Paper 3’s extensions express the same architectural principle Paper 1 established — governance authority is in the authored substrate content, not in the

implementation through which that content is held or used. Paper 3 neither replaces nor qualifies that principle; it carries it to a scope Paper 1 did not address.

2. Source Commitment at Paper 1 Scope

Paper 1, Claim 5 states that the CKS architecture is tool-agnostic: any environment satisfying three minimal requirements can host a CKS substrate, and no environment failing any one of them can. The three requirements — persistent structured state, direct human read/write access, and LLM access to substrate content — are jointly sufficient and individually necessary. No other environmental capability is required.

The architectural significance of this commitment is precise. Because the three requirements are satisfied by commodity tools — spreadsheets, document editors, wikis, structured-field project systems — governance is exercisable by non-specialists in familiar environments. The requirements describe a host interface, not a particular implementation. A substrate hosted in a spreadsheet, a relational database, and a structured document store are the same architectural object, provided all three environments meet the same interface specification. A substrate migrated from one compliant host to another remains the same substrate; migration safety is architecturally guaranteed by the host interface specification rather than by platform-specific engineering.

What this commitment establishes at intra-Self scope is a clean separation between governance content and governance host. The governance authority — the decisions encoded in substrate content, the orchestration rules humans have authored, the conflicts preserved as substrate-level state — belongs to the content layer. The host layer is interchangeable within the space defined by the three requirements. Binding the substrate to a particular host beyond those requirements may be a sound deployment decision; what it cannot be is the source of governance authority. The architecture locates that authority in the content, and the content is what persists across any compliant host migration.

Paper 1 states this commitment at intra-Self scope: each Self’s governance substrate, each cell’s coordination content, can be hosted on any compliant platform. At this scope, tool agnosticism applies to the technical infrastructure a single organization controls for its own CKS deployment.

3. Extension to Inter-Self Scope

Paper 3 extends the CKS architecture to inter-Self coordination, where two or more CKS-governed Selves — each with its own home governance perimeter, its own organizational authority structure, and its own technical substrate infrastructure — coordinate through a shared substrate constructed temporarily for an interaction. Two distinct extensions of tool agnosticism operate at this scope.

3.1 Shared Substrate Platform Independence

When two CKS-governed Selves coordinate, the shared substrate through which they do so is itself a CKS substrate. Paper 3, Claim 1 establishes that the shared substrate carries all six Paper 1 architectural

commitments within scope. Tool agnosticism is one of those six commitments. Its application at the shared substrate level has a specific form that goes beyond its Paper 1 scope: the shared substrate need not use the same technical platform as either participating Self's home substrate.

This matters because at inter-Self scope, two distinct organizations are involved, each potentially running its home substrate on different infrastructure. One Self may host its substrate in a relational database; the other in a structured document store. The shared substrate they construct for a coordination event is not required to match either. Any environment satisfying the three minimal requirements — persistent structured state, direct human read/write access, LLM access to substrate content — can host the shared substrate. The governance content for the interaction — configuration specifying which aspects each Self contributes, conflict registry, contribution records, persistence policy — is hosted in that compliant environment regardless of what technical choices each Self made independently for its home infrastructure.

The architectural principle is the same one Paper 1 established at intra-Self scope: governance authority is in the substrate content, not in the platform hosting it. At inter-Self scope, this means joint governance authority — spanning more than one organization's governance perimeter — is likewise in the shared substrate content, not in the joint technical infrastructure used to host it. Two organizations do not need to agree on a common technical platform to achieve CKS-governed inter-Self coordination. They need to agree on a compliant host for the shared substrate: any environment meeting the three requirements qualifies.

This is tool agnosticism applied to joint technical infrastructure. Paper 1 established it at the level of a single Self's deployment; Paper 3 carries it to the inter-organizational level where multiple Selves' governance perimeters span a single shared substrate.

3.2 Domain Agnosticism as the Content Analog

Paper 3 introduces a second form of agnosticism that did not appear at Paper 1's intra-Self scope: domain agnosticism. The shared-substrate architecture and the Full Aspect Integration mechanism through which Selves exchange content are designed to operate regardless of what organizational domain the participating Selves inhabit. Two Selves in a healthcare context, two in finance, two in operations, or two in entirely different industries with different regulatory environments and different content conventions can each participate in the same CKS inter-Self coordination architecture without the architecture itself requiring modification.

Paper 3's thought experiment is domain-agnostic by construction: the two participating Selves are described by the architectural shape of their coordination situations rather than by their domain content. Paper 3 makes explicit that anchoring the thought experiment to a specific domain would shrink the architectural claim to that domain, since the claim is about cross-organizational coordination across distinct authority structures, not about one cross-organizational instance. The domain-agnostic posture is architecturally derived from the inter-Self scope of Paper 3's claim.

Domain agnosticism is the content analog of tool agnosticism. The parallel is structural:

Tool agnosticism says: the architecture works on any compliant technical platform, because governance is in the substrate content rather than in the platform. Governance requirements do not prescribe a specific database, document format, or runtime environment; they are met through the three minimal host-interface requirements.

Domain agnosticism says: the architecture works in any organizational domain, because governance requirements are addressed through substrate configuration rather than through architectural modification. Domain-specific content — the regulated protocols of a healthcare context, the reporting requirements of a financial context, the operational patterns of a logistics context — is expressed as substrate content under human-governed authorship, not as architectural specialization.

In both cases, the variable dimension (technical platform; organizational domain) is handled by what is authored into the substrate rather than by changing what the architecture is. Governance is in the content; the implementation — whether technical or organizational — is external to the architecture's core commitment.

4. What Is Preserved and What Is Extended

What is preserved from Paper 1: The architectural principle that governance authority is in the substrate content, not in the platform or implementation hosting it. The three minimal requirements for a compliant host remain unchanged. The commitment that the architecture works across any compliant technical environment — without requiring a specific stack, database, or runtime — carries through to inter-Self scope without modification.

What is extended at inter-Self scope: Two new dimensions that were not present at Paper 1's intra-Self scope:

First, the shared substrate's technical platform is added as a scope of tool agnosticism. At intra-Self scope, tool agnosticism governed each Self's home substrate deployment. At inter-Self scope, it also governs the joint technical infrastructure supporting the shared substrate — and specifically licenses that infrastructure to differ from either participating Self's home deployment.

Second, domain agnosticism is introduced as the organizational content analog of tool agnosticism. At intra-Self scope, no cross-domain coordination question arises; a single Self operates within its own organizational context. At inter-Self scope, participating Selves may serve different domains with different content conventions and regulatory requirements. Domain agnosticism is the architectural commitment that this does not require the architecture to be domain-specialized: the same pattern operates through governance-configured substrate content across any domain pair.

The underlying principle — governance in the content, not in the implementation — is the same in both cases. The extensions are extensions of scope, not modifications of the principle.

5. Prior-Art Inheritance Claim

The inheritance chain from Paper 3 back to Paper 1 is direct for both extensions.

Shared substrate platform independence is tool agnosticism applied to the joint technical infrastructure of inter-Self coordination. Any implementation that commits to hosting a shared coordination substrate on any platform satisfying the three minimal requirements — regardless of what platforms the participating organizations use for their respective home substrates — is implementing the Paper 1 tool-agnosticism commitment at inter-Self scope. The three requirements are the same requirements; the scope is the shared substrate rather than a single Self’s home substrate.

Domain agnosticism is the content analog of tool agnosticism extended to the organizational domain dimension. Any implementation that commits to operating the same inter-organizational coordination architecture across distinct organizational domains — expressing domain-specific governance requirements as substrate configuration rather than as architectural specialization — is implementing the principle Paper 1 established for the technical dimension, now applied to the organizational dimension. The principle is the same: governance is in the content; the variable external dimension is handled through authorship.

Both extensions are therefore prior-art-closed by Paper 1’s tool-agnosticism commitment and its formalization in the derivation-note series. Any party seeking to claim novelty for either shared-substrate platform independence or domain-agnostic inter-organizational coordination architecture in a CKS-compatible pattern would need to identify a meaningful distinction from the inheritance this note formalizes. The architectural content — governance in the content, any compliant host for the shared substrate, governance requirements addressed through configuration in any organizational domain — is established prior art as of Paper 1’s publication date.

6. Operational Test

An inter-Self coordination architecture passes the tool-agnosticism inheritance test at this scope if and only if:

1. The shared substrate can be hosted on any environment satisfying Paper 1’s three minimal requirements — persistent structured state, direct human read/write access, LLM access to substrate content — and no additional environmental capability is required as a precondition of shared substrate operation.
2. The shared substrate is not required to use the same technical platform as either participating Self’s home substrate. Two Selves hosting their respective home substrates on different compliant platforms can construct a shared substrate on a third compliant platform without architectural

failure.

3. The governance configuration for a coordination event — specifying contributing aspects, conflict-handling rules, persistence policy, and joint authority structure — is expressed as authored substrate content, not as platform-specific configuration of the hosting environment.

4. The architecture operates without modification across distinct organizational domains. Domain-specific governance requirements — regulatory compliance, content conventions, authority structures — are addressed through governance-authored substrate content, not through architectural variation per domain.

An implementation failing condition (1) or (2) does not satisfy the tool-agnosticism inheritance. An implementation failing condition (3) has located governance authority in the platform rather than the content. An implementation failing condition (4) has not inherited domain agnosticism as the content analog of tool agnosticism.

Source Papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Inter-Self Coordination via Shared Substrate / Full Aspect Integration*. April 2026. ORCID: 0009-0004-8065-3235.

How to Cite This Note

Li, W. (2026). *P3↔P1 Tool Agnosticism Inheritance*. May 15, 2026. ORCID: 0009-0004-8065-3235.